

Chapter 7

Worksheet

Instructions:

Please complete all activities in this worksheet. Write your answers in the spaces provided or on separate sheets if needed.

Worksheet for Chapter 7: Exploring JavaScript for Interactive Apps

Instructions for Students: Complete each activity in this worksheet by following the steps provided. Use a text editor (like Notepad, VS Code, or an online editor like CodePen) to write and test your code. Save your files with the specified names for easy reference. Be creative, experiment with the code, and don't be afraid to make mistakes—it's part of learning! Share your completed projects with classmates or your teacher for feedback. Let's dive into coding!

Activity 1: Experiment with JavaScript Basics

Objective: Get comfortable with basic JavaScript interactivity by modifying simple examples. **Task:**

- Create a new HTML file named `myAlert.html`. Write code to display a custom alert message when the page loads. Change the message to something personal or funny, like "Hey, I'm becoming a coder!"
- Create another HTML file named `textSwap.html`. Build a webpage with a paragraph of text and a button. When the button is clicked, the text should change to something else (e.g., from "Hello!" to "JavaScript rocks!"). Use `document.getElementById()` and `innerHTML` to achieve this.
- Add a second button to `textSwap.html` that changes the text back to the original message or to a new funny phrase. **Deliverable:** Save both files and test them in a browser. Write a short note (2-3 sentences) in a separate text file named `activity1_notes.txt` about what you learned from changing the text dynamically and how alerts differ from on-page text updates.

Activity 2: Build a Personalized Greeting Generator

Objective: Practice using variables and events to create a user-driven interactive feature. **Task:**

- Create a new HTML file named `greetingGen.html`. Include an input field for the user to enter their name, a button to trigger a greeting, and a paragraph where the greeting will appear.
- Write JavaScript code to store the user's input in a variable and display a personalized greeting (e.g., "Hey, [Name]! Welcome to my awesome page!") when the button is clicked.
- Add a CSS style to make the greeting text colourful (e.g., red or blue) and centered on the page. Bonus: Add a hover effect to the button to change its background colour. **Deliverable:** Save the file and test it in a browser with different names. Take a screenshot of the page showing a greeting with your name and save it as `activity2_screenshot.png`.

Activity 3: Enhance the Simple Calculator

Objective: Extend the calculator project from the chapter by adding new features and error handling. **Task:**

- Open or recreate the `calculator.html` file from Chapter 7 (if not saved, refer to the chapter for the code). Ensure it can perform addition, subtraction, multiplication, and division.

- Add a “Clear” button that resets both input fields to empty and the result text to “Result will show here!”
- Add a new feature to calculate the square of the first number (ignore the second input for this). Create a button labeled “Square (x²)” that displays the result as “Square of [num1] is [result]”.
- Style the new “Square” button with a different background colour (e.g., purple) to distinguish it from the other operation buttons. **Deliverable:** Save the updated calculator.html file. Test all operations, including the new square function and clear button, in a browser. Write a short note in a file named activity3_notes.txt (2-3 sentences) about how adding the square function differed from the other operations.

Activity 4: Customize the Rock-Paper-Scissors Game

Objective: Personalize the Rock-Paper-Scissors game by adding visual elements and custom messages. **Task:**

- Open or recreate the rpsgame.html file from Chapter 7 (refer to the chapter if needed). Ensure the game works with score tracking and a reset button.
- Add visual flair by including emojis or small images next to each choice button (e.g., 🪨 for Rock, 📄 for Paper, ✂ for Scissors). You can use Unicode emojis directly in the HTML or find free images online and add them with tags (keep image width small, like 30px).
- Customize the result messages to include funny or meme-inspired feedback (e.g., “You win! You’re a total Rockstar!” for a Rock victory, or “Computer wins! Got snipped by Scissors, ouch!”). **Deliverable:** Save the updated rpsgame.html file. Play a few rounds in a browser to test the new visuals and messages. Save a screenshot of a game result showing the emojis/images and custom message as activity4_screenshot.png.

Activity 5: Create a Simple Colour Changer App

Objective: Combine HTML, CSS, and JavaScript to build a new mini-project that changes the page background colour based on user input. **Task:**

- Create a new HTML file named colourChanger.html. Add a title (“Colour Changer App”), a set of three buttons labelled “Red”, “Blue”, and “Green”, and a paragraph that says “Click a button to change the background!”
- Write JavaScript code to change the page’s background colour (document.body.style.backgroundColor) to the corresponding colour when each button is clicked (e.g., clicking “Red” sets the background to red).
- Add CSS to style the buttons with matching background colours (e.g., the “Red” button has a red background) and white text for visibility. Include a hover effect to darken the button colour slightly.
- Bonus: Add a fourth button labelled “Random” that picks a random colour from a list (e.g., use an array of colours like ["yellow", "purple", "orange"] and Math.random() to select one). **Deliverable:** Save colourChanger.html and test all buttons in a browser. Write a short note in a file named activity5_notes.txt (2-3 sentences) about how you added the random colour feature (if attempted) or what you found most interesting about changing styles with JavaScript.

Activity 6: Brainstorm and Plan Your Own Mini-Project

Objective: Apply your JavaScript skills creatively by designing a unique interactive app or game concept. **Task:**

- Think of a small project idea that uses interactivity (e.g., a “Mood Picker” that changes an image based on how you feel, a “Random Joke Generator” with a button to show jokes, or a “Click Counter” game to count button clicks in 10 seconds).
- Write a plan in a text file named activity6_plan.txt that includes:
 - The project name and a 1-2 sentence description of what it does.
 - A list of HTML elements needed (e.g., buttons, text areas).
 - A list of JavaScript features to use (e.g., variables, events, conditionals).
 - Ideas for CSS styling (e.g., colours, layout).
- If time permits, start coding your idea in a new HTML file named after your project (e.g., moodPicker.html). Even if incomplete, save your progress. **Deliverable:** Submit activity6_plan.txt with your detailed plan. If you

started coding, include the HTML file as well. Be ready to discuss your idea with classmates or your teacher for feedback.

Completion Checklist:

- Activity 1: myAlert.html, textSwap.html, activity1_notes.txt
- Activity 2: greetingGen.html, activity2_screenshot.png
- Activity 3: Updated calculator.html, activity3_notes.txt
- Activity 4: Updated rpsgame.html, activity4_screenshot.png
- Activity 5: colourChanger.html, activity5_notes.txt
- Activity 6: activity6_plan.txt, optional project HTML file

Final Note for Students: This worksheet is your chance to play with JavaScript and build cool things! Take your time with each activity, test your code often, and add your personal touch to make projects fun. If you get stuck, refer to Chapter 7 or ask for help. Keep coding and creating—you're on your way to becoming a web development pro!